RELATÓRIO DE ARQUITETURA DE SOFTWARE

Projeto: CashFlow API
Teste para Entrevista

Nome do Arquiteto de Software: Victor Caetano

---

1. Problema a Ser Resolvido

O sistema proposto tem como objetivo gerenciar operações financeiras de entrada e saída (créditos e débitos), garantindo:

- Consistência dos dados financeiros
- Processamento confiável mesmo em cenários de falha
- Escalabilidade para alto volume de transações

Além disso, o sistema deve permitir a consolidação eficiente de saldos diários sem impactar a performance das operações transacionais.

---

2. Objetivo do Projeto

Construir um sistema de controle de fluxo de caixa altamente escalável, resiliente e observável, utilizando práticas modernas de arquitetura de software, incluindo:

- Event-Driven Architecture
- CQRS (Command Query Responsibility Segregation)
- Outbox Pattern
- Processamento assíncrono com mensageria
- Observabilidade completa

---

3. Solução Arquitetural Proposta

Foi adotada uma arquitetura baseada em eventos, com separação clara entre processamento transacional e processamento assíncrono.

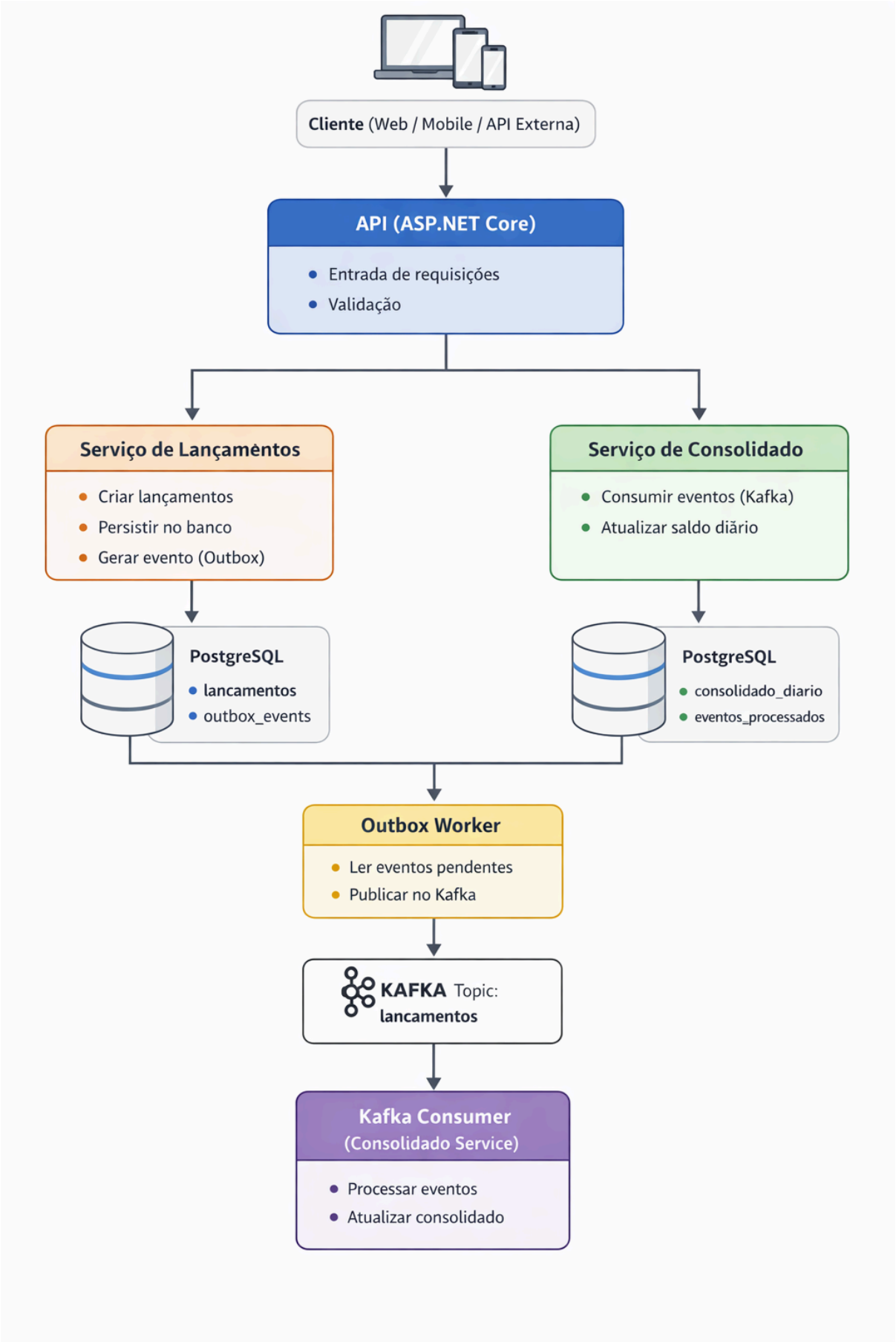
Fluxo principal:

1. A API recebe requisições de lançamento
2. Os dados são persistidos no banco transacional (PostgreSQL)
3. Um evento é registrado via Outbox Pattern
4. Um worker publica o evento no Kafka
5. Consumidores processam o evento de forma assíncrona
6. O saldo consolidado é atualizado em um modelo de leitura otimizado

Essa abordagem garante desacoplamento entre componentes, alta resiliência e capacidade de escalabilidade horizontal.

---

4. Arquitetura Geral


Cliente (Web / Mobile / API Externa)
API (ASP.NET Core)
Entrada de requisições
Validação
Serviço de Lançamentos
Criar lançamentos
Persistir no banco
Gerar evento (Outbox)
Serviço de Consolidado
Consumir eventos (Kafka)
Atualizar saldo diário
PostgreSQL
lancamentos
outbox_events
PostgreSQL
consolidado_diario
eventos_processados
Outbox Worker
Ler eventos pendentes
Publicar no Kafka
KAFKA Topic:
lancamentos
Kafka Consumer
(Consolidado Service)
Processar eventos
Atualizar consolidado

Componentes principais:

API (ASP.NET Core 8)

- Responsável por receber requisições HTTP
- Validação e regras de negócio
- Persistência de dados
- Geração de eventos

Worker (Background Service)

- Processa eventos de forma assíncrona
- Consome mensagens do Kafka
- Atualiza o modelo de leitura (consolidado diário)

PostgreSQL (AWS RDS)

- Fonte de verdade dos dados
- Armazena lançamentos, eventos e consolidado

Kafka (KRaft Mode – sem Zookeeper)

- Responsável pela mensageria
- Garante desacoplamento entre serviços
- Permite múltiplos consumidores

Prometheus

- Coleta métricas da aplicação

Grafana

- Visualização gráfica das métricas
- Dashboards operacionais

---

5. Fluxo de Dados

1. Cliente envia requisição → `POST /lancamentos`
2. API grava dados no PostgreSQL
3. Evento é registrado na tabela de outbox
4. Worker lê eventos pendentes
5. Worker publica evento no Kafka
6. Consumidor processa o evento
7. Atualização do `consolidado_diario`
8. Métricas são expostas via `/metrics`
9. Prometheus coleta métricas
10. Grafana exibe dashboards

---

6. Requisitos Funcionais

O sistema atende aos seguintes requisitos:

- **RF01:** Permitir o registro de lançamentos financeiros (crédito e débito)
- **RF02:** Armazenar todos os lançamentos de forma imutável
- **RF03:** Gerar eventos a cada lançamento realizado
- **RF04:** Processar eventos de forma assíncrona
- **RF05:** Consolidar o saldo diário com base nos eventos processados
- **RF06:** Garantir idempotência no processamento de eventos
- **RF07:** Permitir consulta de saldo consolidado por data

---

7. Requisitos Não Funcionais

Escalabilidade

- Uso de Kafka permite múltiplos consumidores
- Processamento distribuído via workers

Resiliência

- Retry com backoff exponencial
- Circuit breaker para falhas
- Dead Letter Queue para falhas persistentes

Consistência

- Consistência eventual baseada em eventos
- Outbox Pattern evita perda de dados

Performance

- Separação entre escrita e leitura (CQRS)
- Consultas otimizadas no modelo de leitura

Observabilidade

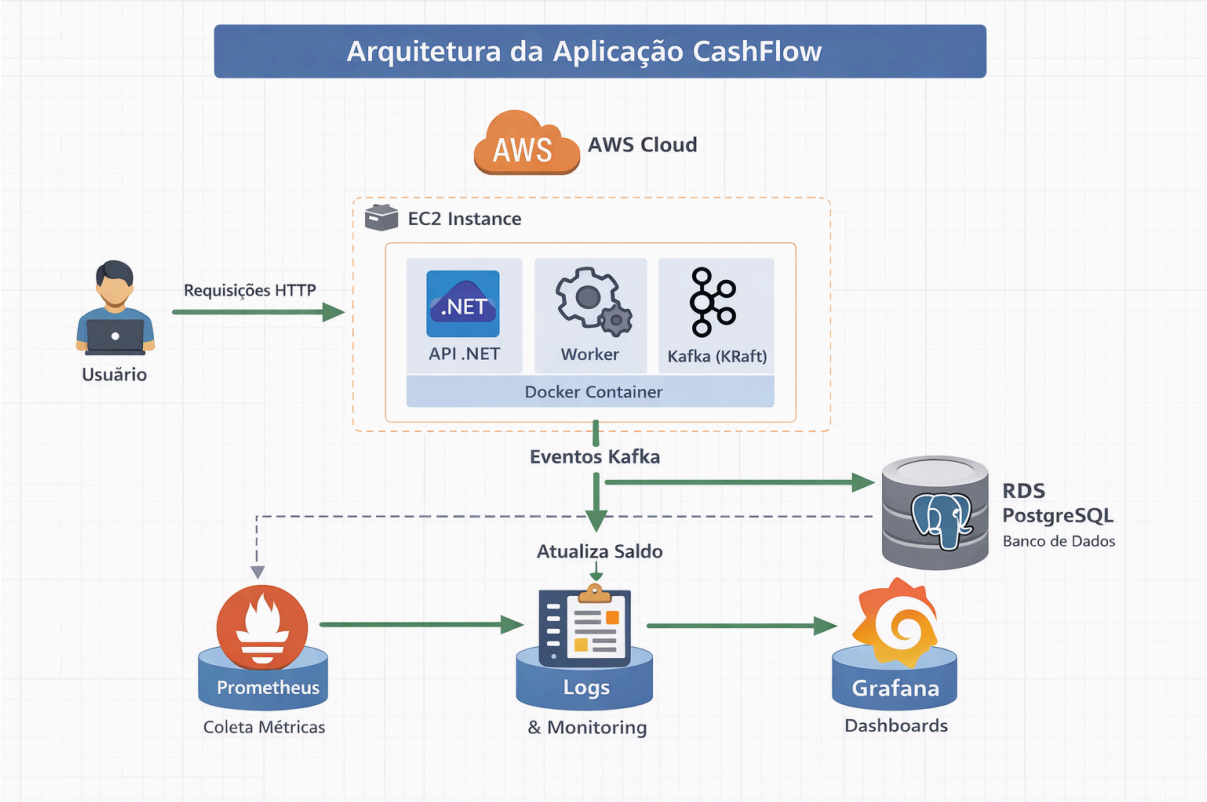
- Logs estruturados com Serilog
- Health Checks reais
- Métricas com OpenTelemetry
- Monitoramento com Prometheus e Grafana

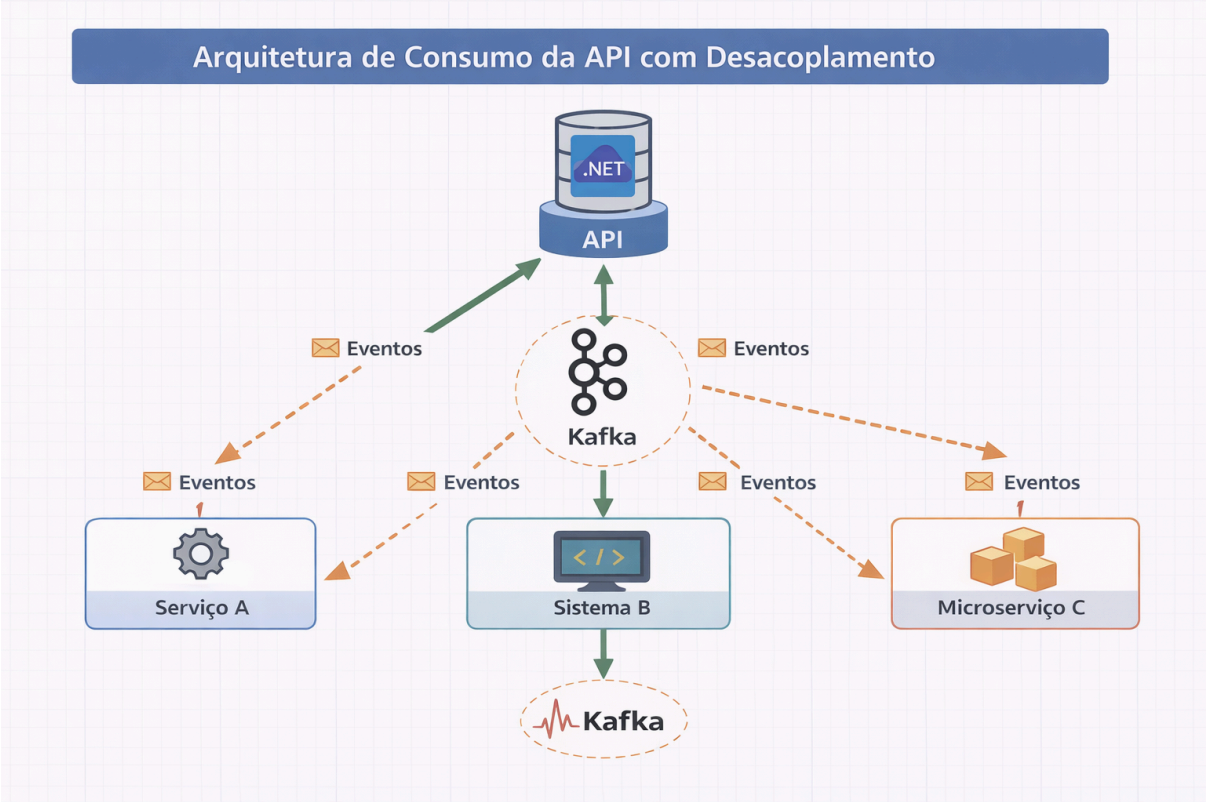
Manutenibilidade

- Separação de responsabilidades
- Uso de princípios SOLID
- Clean Architecture

---

8. Decisões Arquiteturais





Event-Driven Architecture

Motivo: desacoplamento e escalabilidade
Benefício: permite múltiplos consumidores independentes

Kafka (KRaft Mode)

Motivo: alta performance e arquitetura moderna
Benefício: elimina dependência do Zookeeper

Outbox Pattern

Motivo: garantir consistência entre banco e eventos
Benefício: evita perda de eventos

CQRS

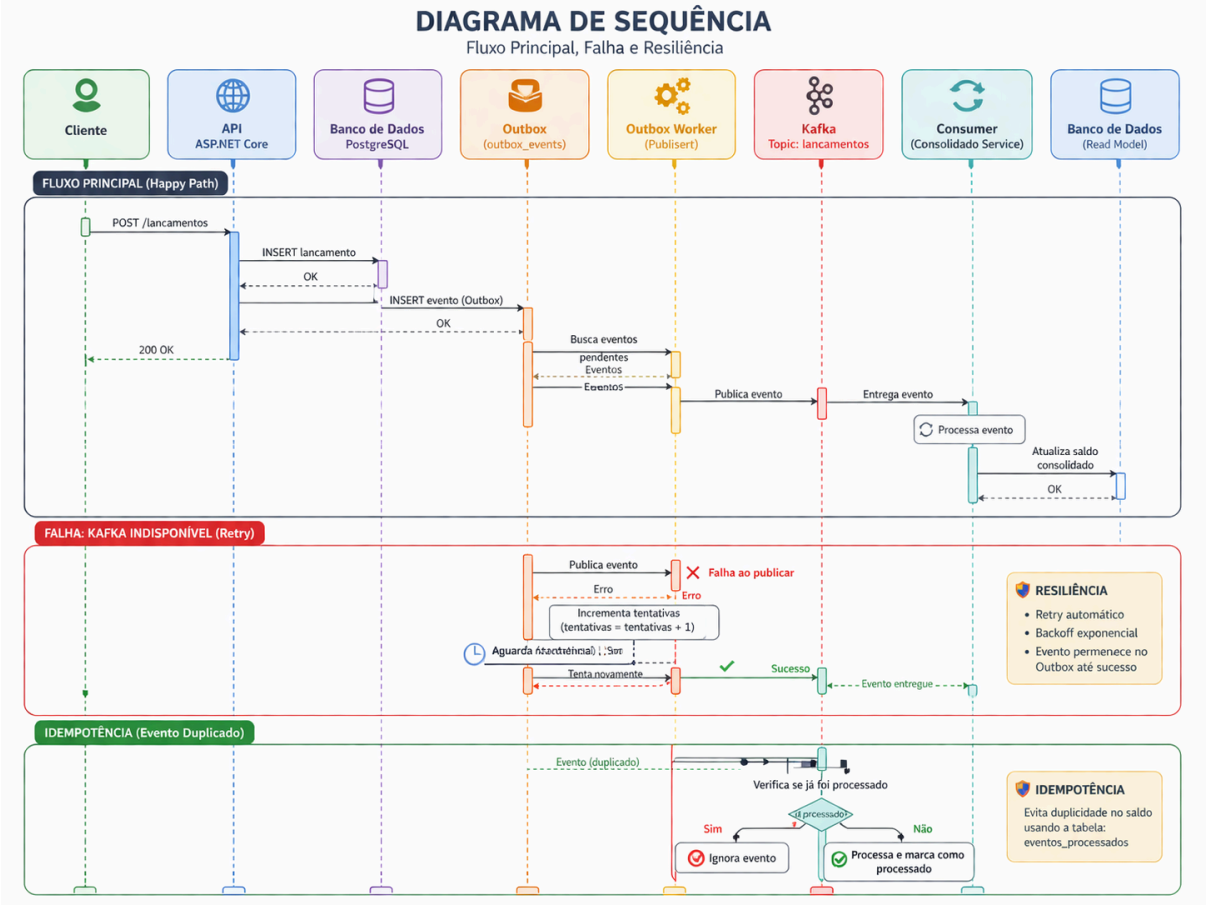
Motivo: separar leitura e escrita
Benefício: ganho de performance e escalabilidade

Containerização com Docker

Motivo: portabilidade e padronização
Benefício: facilidade de deploy e replicação de ambiente

---

9. Observabilidade Implementada



Logs

- Serilog com logs estruturados

Health Checks

- `/health/live` (liveness)
- `/health/ready` (readiness)

Métricas

- Tempo de resposta HTTP
- Número de requisições
- Uso de memória (.NET GC)
- Threads e conexões

Endpoint

- `/metrics` (Prometheus)

---

10. Containerização

Ambiente completo via Docker Compose contendo:

- API
- Worker
- Kafka (KRaft)
- Prometheus
- Grafana

Benefícios:

- Ambiente replicável
- Facilidade de setup
- Pronto para CI/CD

---

11. Segurança (Inicial)

- Autenticação via JWT
- Configuração via variáveis de ambiente
- Separação de credenciais

---

12. Problemas Resolvidos Durante a Implementação

- Erros de injeção de dependência (lifecycle incorreto)
- Problemas de conexão com banco de dados
- Ajustes em Dockerfiles e build
- Conflitos de versões de dependências
- Integração de métricas com OpenTelemetry
- Configuração de Kafka sem Zookeeper

---

13. Roadmap (Evolução)

Alta Prioridade

- Dead Letter Queue integrada ao Kafka
- Política de retenção e particionamento
- Dashboards Grafana completos

Médio Prazo

- Cache com Redis
- Rate limiting
- Versionamento de eventos

Produção

- Deploy em AWS (ECS ou Kubernetes)
- Kafka gerenciado (MSK)
- Observabilidade avançada (Grafana Cloud / Datadog)
- Tracing distribuído

---

14. Evoluções Futuras

- Arquitetura de microserviços independentes
- Multi-tenant
- Event sourcing completo
- Machine learning para análise financeira

---

15. Conclusão

A solução proposta atende de forma eficiente aos requisitos funcionais e não funcionais, utilizando padrões modernos amplamente adotados em sistemas distribuídos e financeiros.

Os principais destaques da arquitetura incluem:

- Desacoplamento entre componentes
- Alta tolerância a falhas
- Escalabilidade horizontal
- Observabilidade completa
- Base sólida para evolução futura

A arquitetura foi desenhada priorizando desacoplamento, resiliência e escalabilidade, alinhada com padrões utilizados em sistemas financeiros e ambientes de alta criticidade.